

机器学习导论-*K* 近邻

Introduction to Machine Learning-*KNN*

李文斌

<https://cs.nju.edu.cn/liwenbin>

liwenbin@nju.edu.cn

2024年03月26日

大纲

- 回顾
- k -近邻分类器
- 最近邻分类器
- k -近邻回归
- 降低近邻计算
- 扩展阅读

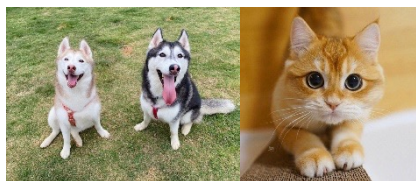
大纲

- 回顾
- k -近邻分类器
- 最近邻分类器
- k -近邻回归
- 降低近邻计算
- 扩展阅读

回顾

□ 统计学角度：机器学习的目的是得到映射 $x \mapsto y$

图像



文本

These existing methods mainly focus on making knowledge transfer [22, 2], concept representation [17, 4] or relation measure [25], but have not paid sufficient attention

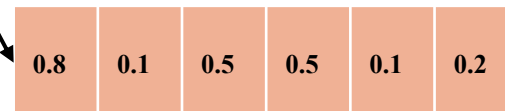
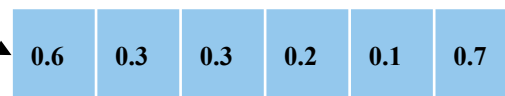
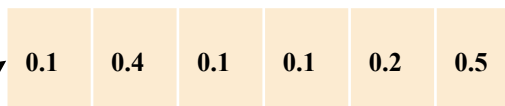
音频



视频



特征工程
手工/自动



特征向量: x

$\Rightarrow y ?$

回顾

□ 特征与属性 (Feature and Attributes)

图像



特征工程
手工/自动

0.2 0.3 0.1 0.6 0.4 0.5

特征向量: $x \in \mathbb{R}^6$

0.2 0.3 0.1 0.6 0.4 0.5

属性1

属性2

...

属性6

回顾

□ 分类问题 Classification

- 训练集: $D_{train} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$
- 训练样本: $\mathbf{x}_i \in X \in \mathbb{R}^d$
- 样本标签: $y_i \in Y = \{1, 2, \dots, C\}$
- 测试集: $D_{test} = \{\bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \dots, \bar{\mathbf{x}}_m\}$

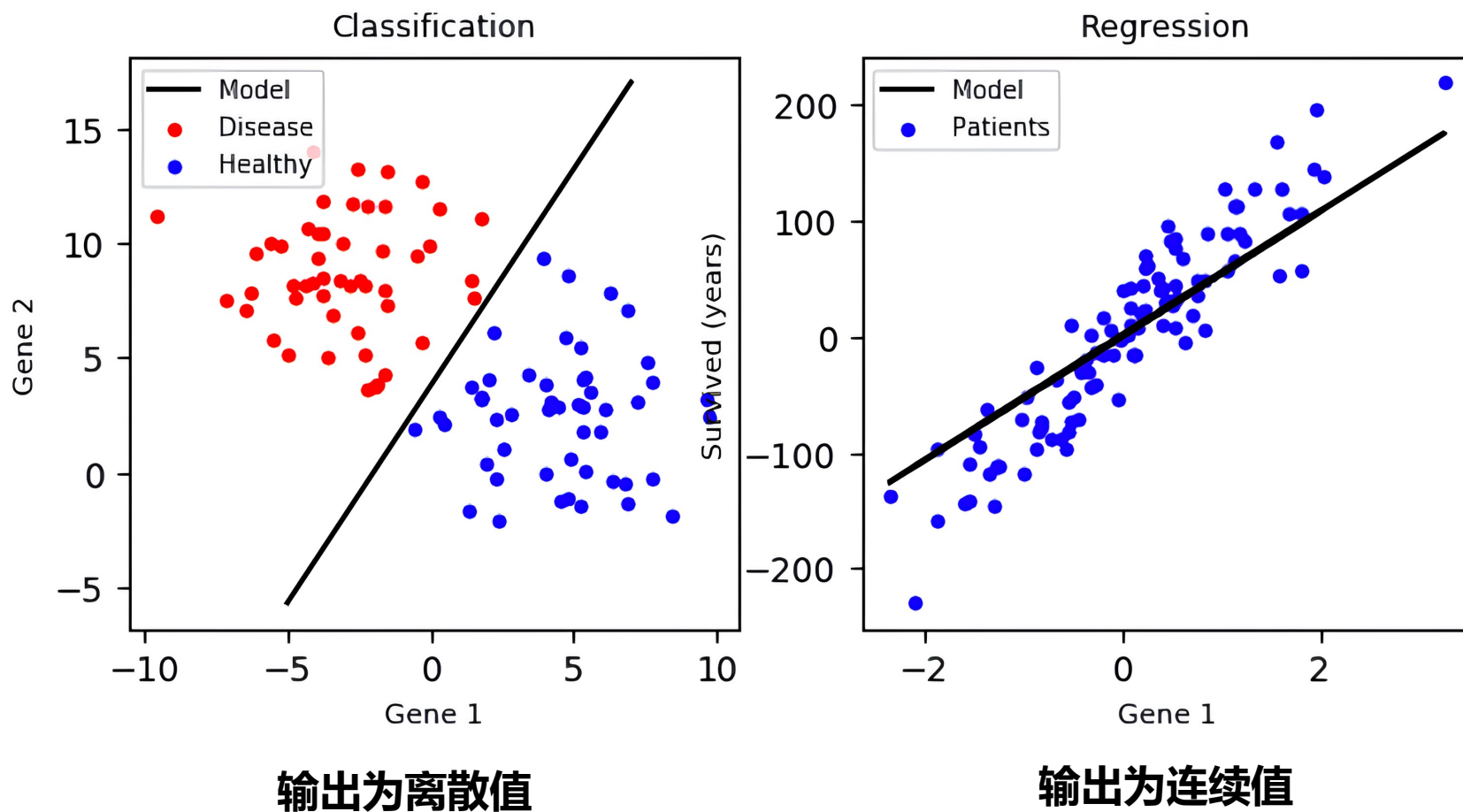
回顾

□ 回归问题 Regression

- 训练集: $D_{train} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$
- 训练样本: $\mathbf{x}_i \in \mathcal{X} \in \mathbb{R}^d$
- 样本标签: $y_i \in \mathbb{R}$
- 测试集: $D_{test} = \{\bar{\mathbf{x}}_1, \bar{\mathbf{x}}_2, \dots, \bar{\mathbf{x}}_m\}$

回顾

□ 分类vs回归



回顾

□ 距离度量 ($x_i, x_j \in \mathbb{R}^d$)

欧氏距离

$$d(x_i, x_j) = \sqrt{(x_i - x_j)^T (x_i - x_j)}$$

余弦相似性

$$s(x_i, x_j) = \frac{x_i^T x_j}{\|x_i\| \|x_j\|}$$

曼哈顿距离

$$d(x_i, x_j) = \|x_i - x_j\|_1$$

切比雪夫距离

$$d(x_i, x_j) = \|x_i - x_j\|_\infty$$

马氏距离

$$d_M(x_i, x_j) = \sqrt{(x_i - x_j)^T M (x_i - x_j)}$$

大纲

- 回顾
- k -近邻分类器
- 最近邻分类器
- k -近邻回归
- 降低近邻计算
- 扩展阅读

K -近邻分类器

□ k -近邻分类器 (k -Nearest Neighbor Classifier, k -NN)

✓ 算法流程

- 计算测试样本 $\bar{x}$ 和 D_{train} 中所有训练样本 x_i 之间的距离 $d(\bar{x}, x_i)$
- 对所有距离值（相似度值）进行升序（降序）排列
- 选择 k 个最近（距离最小/相似度最大）的训练样本
- 采用投票法，将近邻中样本数最多的类别标签分配给 $\bar{x}$

K -近邻分类器

□ k 的取值的影响

- k 一般取奇数值，避免平局
- k 取不同的值，分类结果可能不同
- k 值较小时，对噪声敏感，整体模型变得复杂，容易过拟合
- k 值较大时，对噪声不敏感，整体模型变得简单，容易欠拟合

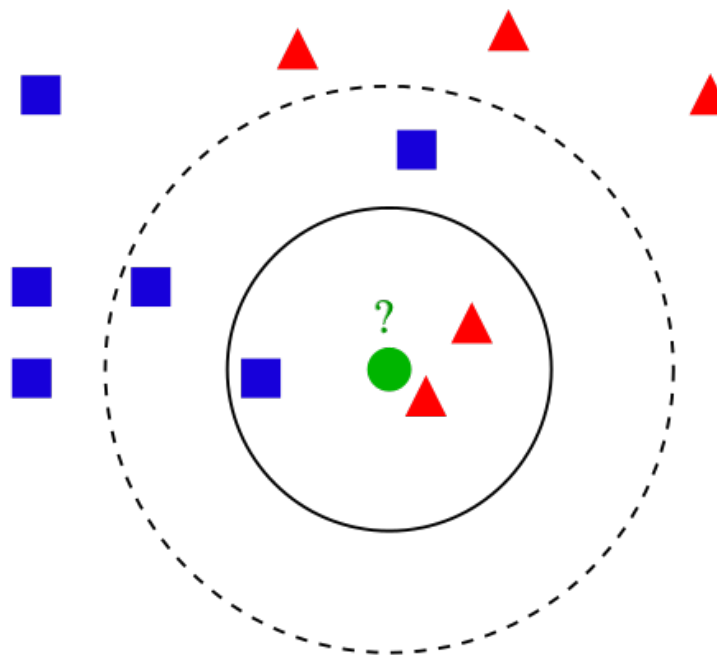
K -近邻分类器

□ k 的取值的影响

■ Class 1

▲ Class 2

● ?



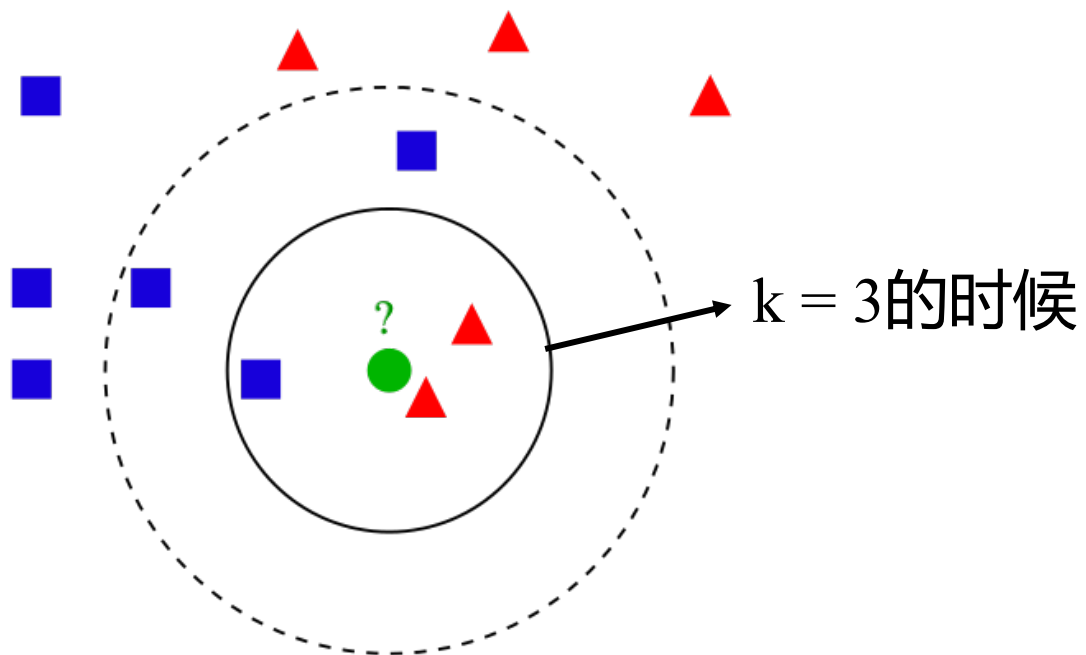
K -近邻分类器

□ k 的取值的影响

■ Class 1

▲ Class 2

● Class 2



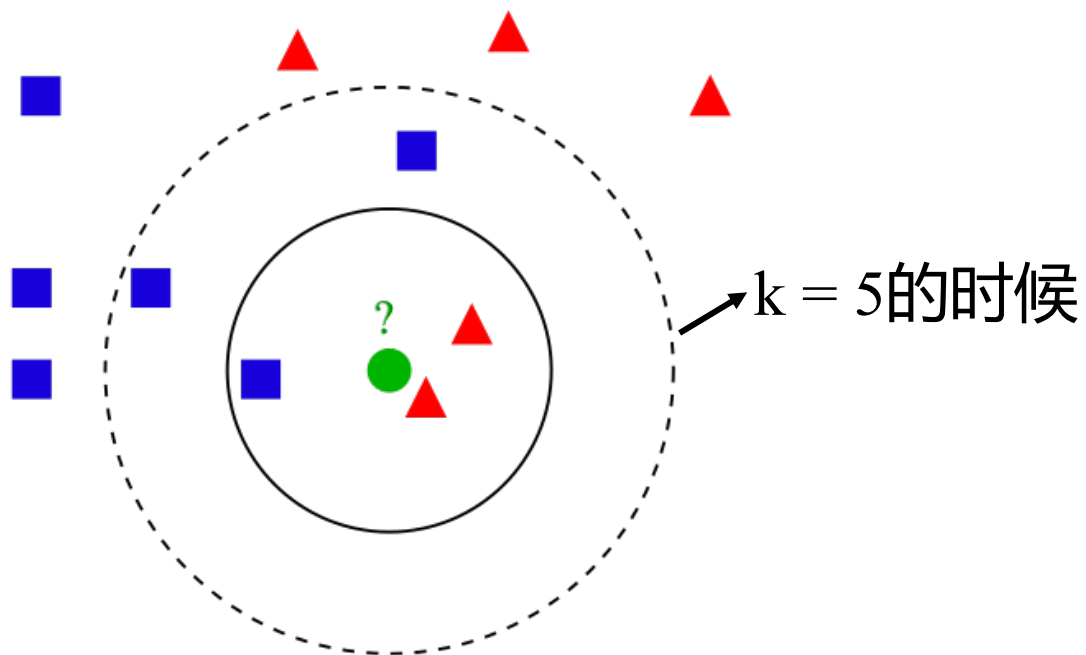
K -近邻分类器

□ k 的取值的影响

■ Class 1

▲ Class 2

● Class 1



大纲

- 回顾
- k -近邻分类器
- 最近邻分类器
- k -近邻回归
- 降低近邻计算
- 扩展阅读

最近邻分类器

□ 最近邻分类器 (1-Nearest Neighbor Classifier, 1-NN)

✓ 算法流程

- 计算测试样本 $\bar{x}$ 和 D_{train} 中所有训练样本 x_i 之间的距离 $d(\bar{x}, x_i)$
- 对所有距离值进行升序排列
- 选择最近的那个训练样本 $i^* = \underset{i}{\operatorname{argmin}} d(\bar{x}, x_i)$
- 将 x_{i^*} 的类别标签分配给 $\bar{x}$

最近邻分类器

□ 泛化错误率

✓ 最近邻分类器的错误率（测试样本为 $\mathbf{x}$ ，其最近邻为 $\mathbf{z}$ ）：

$$P(err) = 1 - \sum_{c \in y} P(c|\mathbf{x})P(c|\mathbf{z})$$

✓ 假设样本i.i.d, 令 $c^* = \operatorname{argmax}_{c \in y} P(c|\mathbf{x})$ 表示贝叶斯最优分类器的结果：

$$\begin{aligned} P(err) &\simeq 1 - \sum_{c \in y} P^2(c|\mathbf{x}) \\ &\leq 1 - P^2(c^*|\mathbf{x}) \\ &= (1 + P(c^*|\mathbf{x}))(1 - P(c^*|\mathbf{x})) \\ &\leq 2 \times (1 - P(c^*|\mathbf{x})) \end{aligned}$$

最近邻分类器

□ 泛化错误率

✓ 最近邻分类器的错误率（测试样本为 $\mathbf{x}$ ，其最近邻为 $\mathbf{z}$ ）：

$$P(err) = 1 - \sum_{c \in y} P(c|\mathbf{x})P(c|\mathbf{z})$$

✓ 假设样本i.i.d, 令 $c^* = \operatorname{argmax}_{c \in y} P(c|\mathbf{x})$ 表示贝叶斯最优分类器的结果：

$$P(err) \simeq 1 - \sum_{c \in y} P^2(c|\mathbf{x})$$

结论：最近邻分类器虽然简单，但它的泛化错误率不超过贝叶斯分类器的错误率两倍！

大纲

- 回顾
- k -近邻分类器
- 最近邻分类器
- k -近邻回归
- 降低近邻计算
- 扩展阅读

K-近邻回归

□ K-近邻回归 (k -Nearest Neighbor Regression)

✓ 算法流程

- 计算测试样本 $\bar{x}$ 和 D_{train} 中所有训练样本 x_i 之间的距离 $d(\bar{x}, x_i)$
- 对所有距离值（相似度值）进行升序（降序）排列
- 选择 k 个最近（距离最小/相似度最大）的训练样本
- 将距离值的倒数作为权重，然后将 k 个近邻的标签值加权平均，作为 $\bar{x}$ 的预测值

K-近邻回归

□ 近邻平滑

✓ 核平滑法 (kernel smoother)

- **二次核** (Epanechnikov quadratic kernel)

$$K_{E,\lambda}(x_0, x) = \begin{cases} 0.75 \left(1 - \frac{(x_0 - x)^2}{\lambda^2} \right) & \text{if } |x - x_0| < \lambda \\ 0 & \text{otherwise} \end{cases}$$

- **次方核** (tricube kernel)

$$K_{T,\lambda}(x_0, x) = \begin{cases} \left(1 - \left| \frac{x - x_0}{\lambda} \right|^3 \right)^3 & \text{if } |x - x_0| < \lambda \\ 0 & \text{otherwise} \end{cases}$$

- **高斯核** (Gaussian kernel)

$$K_{G,\lambda}(x_0, x) = \frac{1}{\lambda\sqrt{2\pi}} e^{-\frac{(x-x_0)^2}{2\lambda^2}}$$

K -近邻分类器

□ 讨论

✓ k -NN是典型的“懒惰学习” (lazy learning)

- 训练阶段仅仅是把样本保存起来，训练时间开销为零，待收到测试样本后再进行处理

✓ SVM、CNN等是“急切学习” (eager learning)

- 在训练阶段就对样本进行学习处理的方法，这类方法尝试在训练期间构造一个通用的，与输入无关的目标函数

K -近邻分类器

□ 讨论

✓ 优点

- 精度高
- 对异常值不敏感
- 无数据输入假定

✓ 缺点

- 计算复杂度高
- 空间复杂度高

K -近邻分类器

□ 讨论

✓ 时间复杂度

- 假设 $d(x_i, x_j)$ 是欧式距离，时间复杂度为 $O(d)$
- 训练阶段：0
- 测试阶段： $O(nd + n\log k)$

✓ 缺点

- 从 n 个数（距离）中选择 k 个最小的，时间复杂度是？
- 空间复杂度？

K-近邻分类器

□ 讨论-排序算法

排序算法	平均时间复杂度	最坏时间复杂度	最好时间复杂度	空间复杂度	稳定性
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
直接选择排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
希尔排序	$O(n \log n)$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	稳定
基数排序	$O(N * M)$	$O(N * M)$	$O(N * M)$	$O(M)$	稳定

K -近邻分类器

□ 讨论

- ✓ 特征维度2-5: 维诺图 Voronoi diagrams
- ✓ 特征维度6-30: KD-Tree
- ✓ 高维特征:
 - 降维算法, 例如PCA
 - 近似最近邻 (approximate nearest neighbor, ANN)
 - 哈希 (hashing)

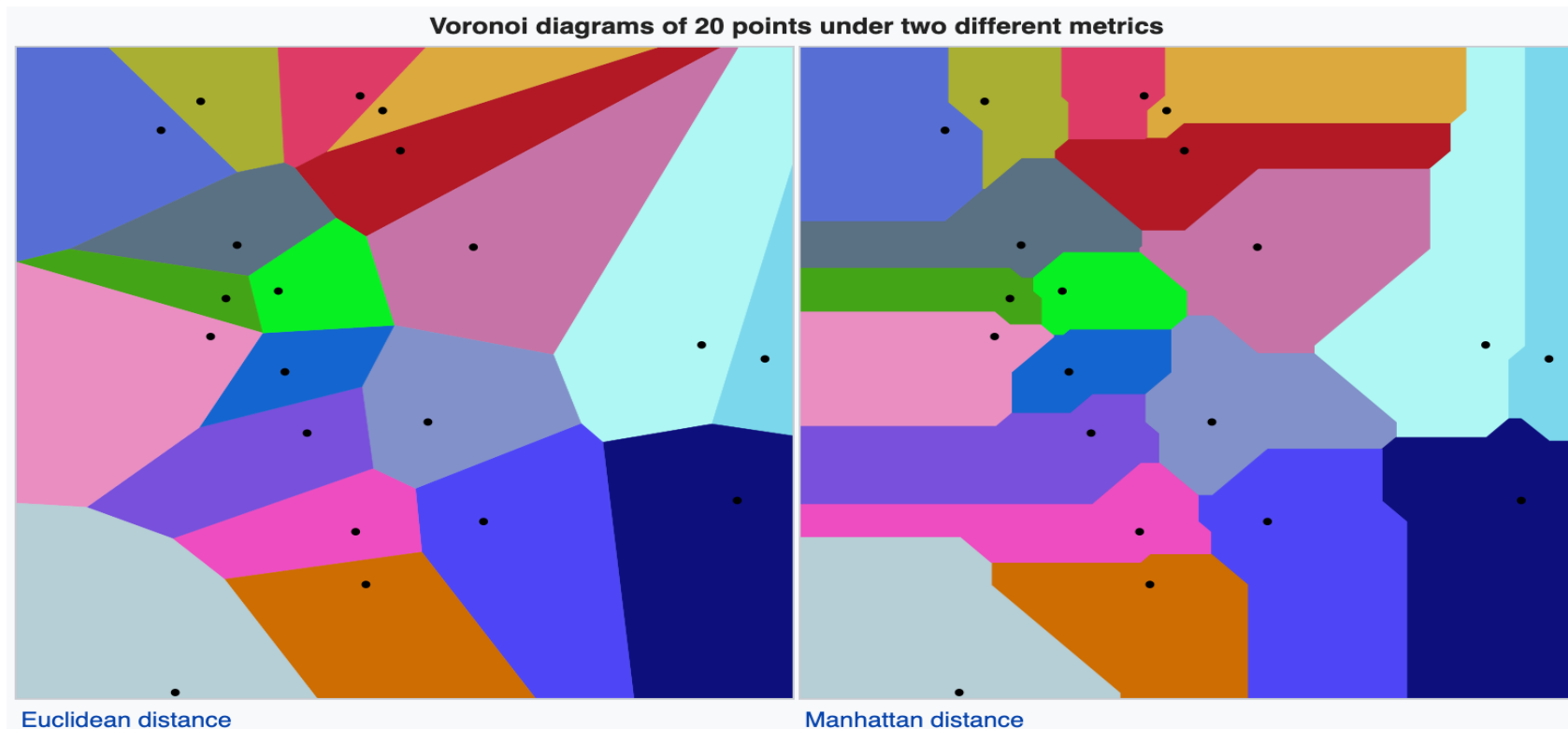
大纲

- 回顾
- k -近邻分类器
- 最近邻分类器
- k -近邻回归
- 降低近邻计算
- 扩展阅读

降低近邻计算

□ 维诺图 (Voronoi diagram)

✓ 定义：根据一组给定的目标，将一个平面划分成靠近每一个目标的多个区块



降低近邻计算

□ 维诺图 (Voronoi diagram)

✓ 维诺图由一系列维诺单元 (Voronoi cells) 组成

假设 X 是一个点集, 包含 K 个基点 $(P_k)_{k \in K}$, 那么维诺单元 R_k 定义为:

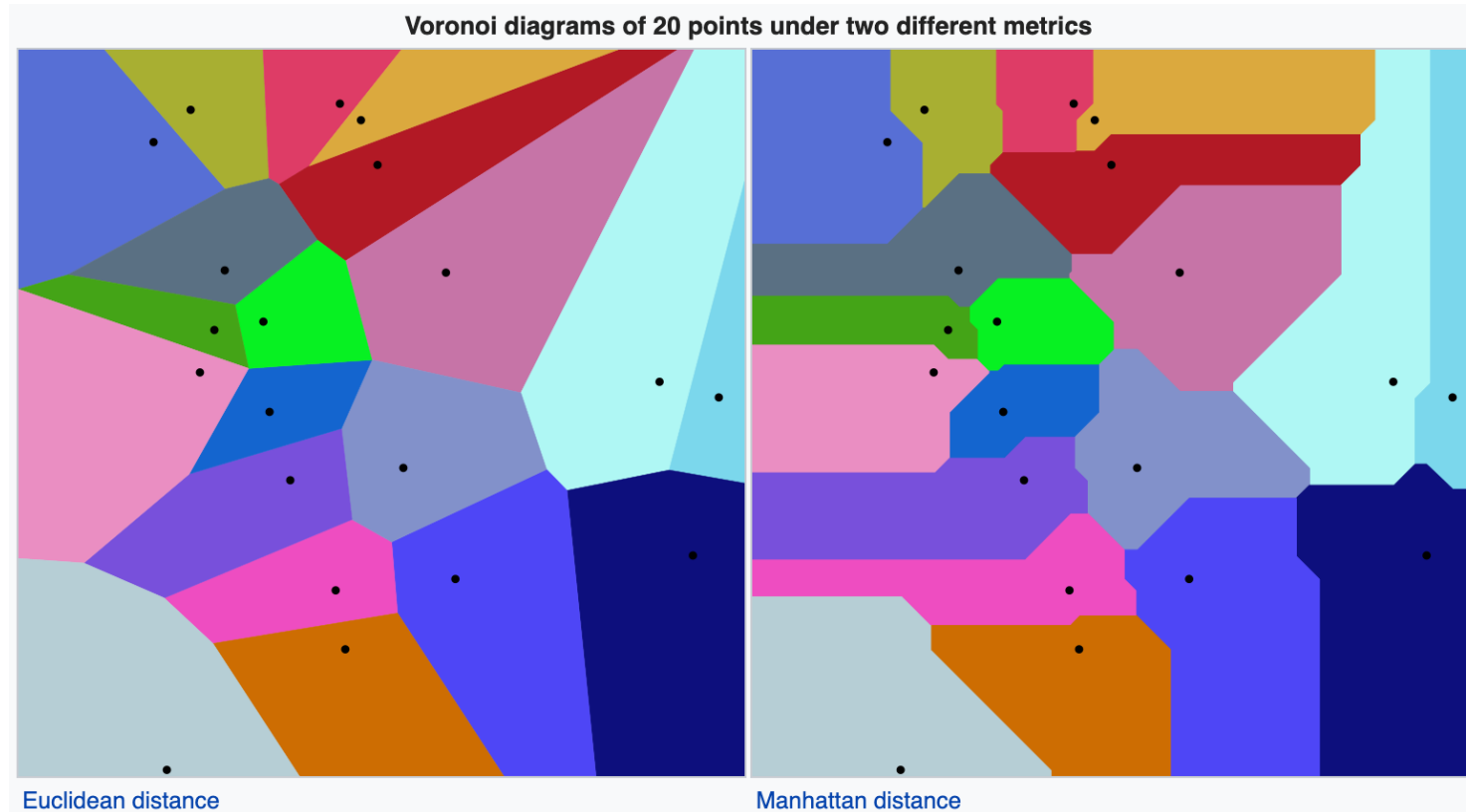
$$R_k = \{x \in X \mid d(x, P_k) \leq d(x, P_j) \text{ for all } j \neq k\}$$

✓ 每个维诺单元永远都是一个凸多面体

降低近邻计算

□ 维诺图 (Voronoi diagram)

✓ 在两种度量下，包含20个点的维诺图：



降低近邻计算

□ 维诺图 (Voronoi diagram)

- ✓ 查询或测试：给定一个查询 $q \in \mathbb{R}^d$ ，找到 $P_k \in X$ ，使得 $q \in R_k$
- ✓ 时间复杂度
 - 2维数据： $O(n \log n)$ 用来计算维诺图和梯形图；查询（测试）时间为 $O(\log n)$ ，其中使用梯形图进行点的定位
 - d 维数据：得使用二叉空间分割树 (binary space partition tree, BSP tree) 进行点的定位，但是时间估计比较难，难以量化
- ✓ 适用范围：1-NN
- ✓ 适合特征维度：2-3维，可能4维或5维

降低近邻计算

□ KD树 (KD-Tree)

- ✓ KD树是一种对K维空间中的实例点进行存储以便对其进行快速检索的树形数据结构。
- ✓ KD树是二叉树，表示对K维空间的一个划分 (partition)。构造KD树相当于不断地用垂直于坐标轴的超平面将K维空间切分，构成一系列的K维超矩形区域。KD树的每个结点对应于一个K维超矩形区域。

降低近邻计算

□ KD树构造

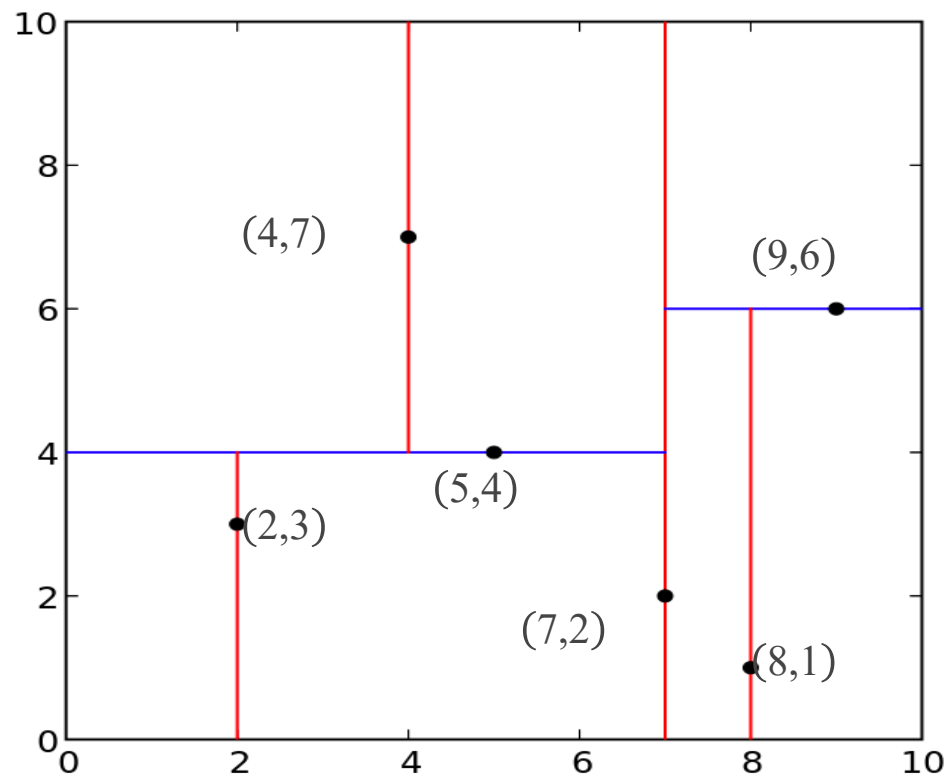
✓ 构造流程:

- 确定split域。计算每个特征维度的方差，方差最大的维度即为split域的值
- 确定Node-data域。数据集点集按其第split域的值排序，中位数的数据点即被选为Node-data
- 对剩下的数据点进行划分，确定左右子空间
- 递归。在每个子空间继续进行空间划分，直到空间中只包含一个数据点

降低近邻计算

□ KD树构造

假设有6个二维数据点 $\{(2,3), (5,4), (9,6), (4,7), (8,1), (7,2)\}$ ，数据点位于二维空间内，如下图黑点所示。

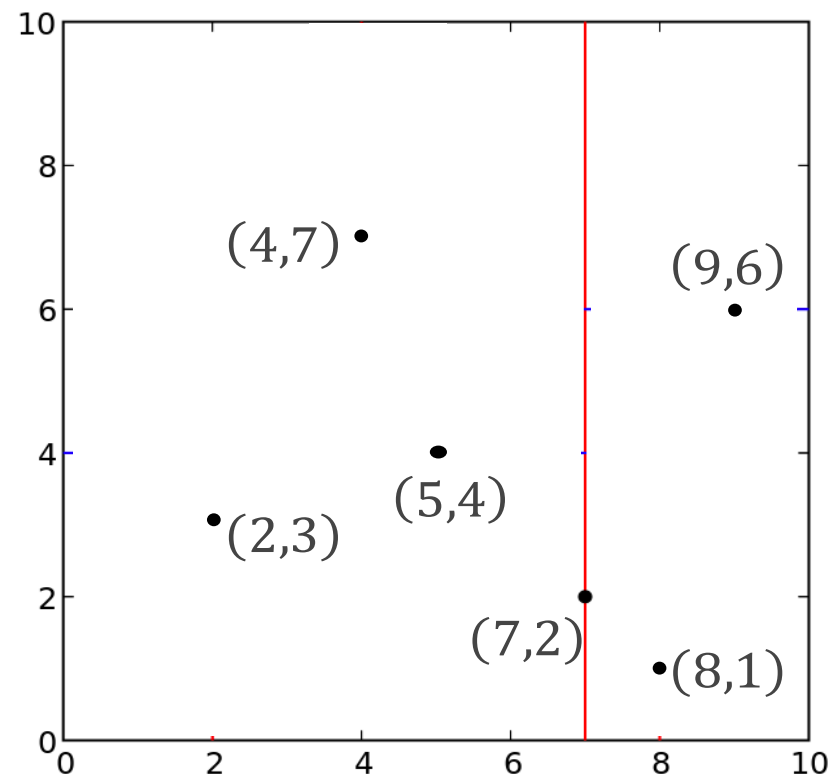


二维数据KD树空间划分示意图

降低近邻计算

□ KD树构造

1. 确定split域的值；分别计算 x , y 方向上的方差，可知 x 方向方差更大，则 $\text{split}=0$ （将 x,y 方向分别编码为0和1）
2. 确定Node-data的值；根据 x 轴方向的值2,5,9,4,8,7得到中位数7，则选定Node-data = (7,2)对空间进行切分，即 $x=7$
3. 递归。。。

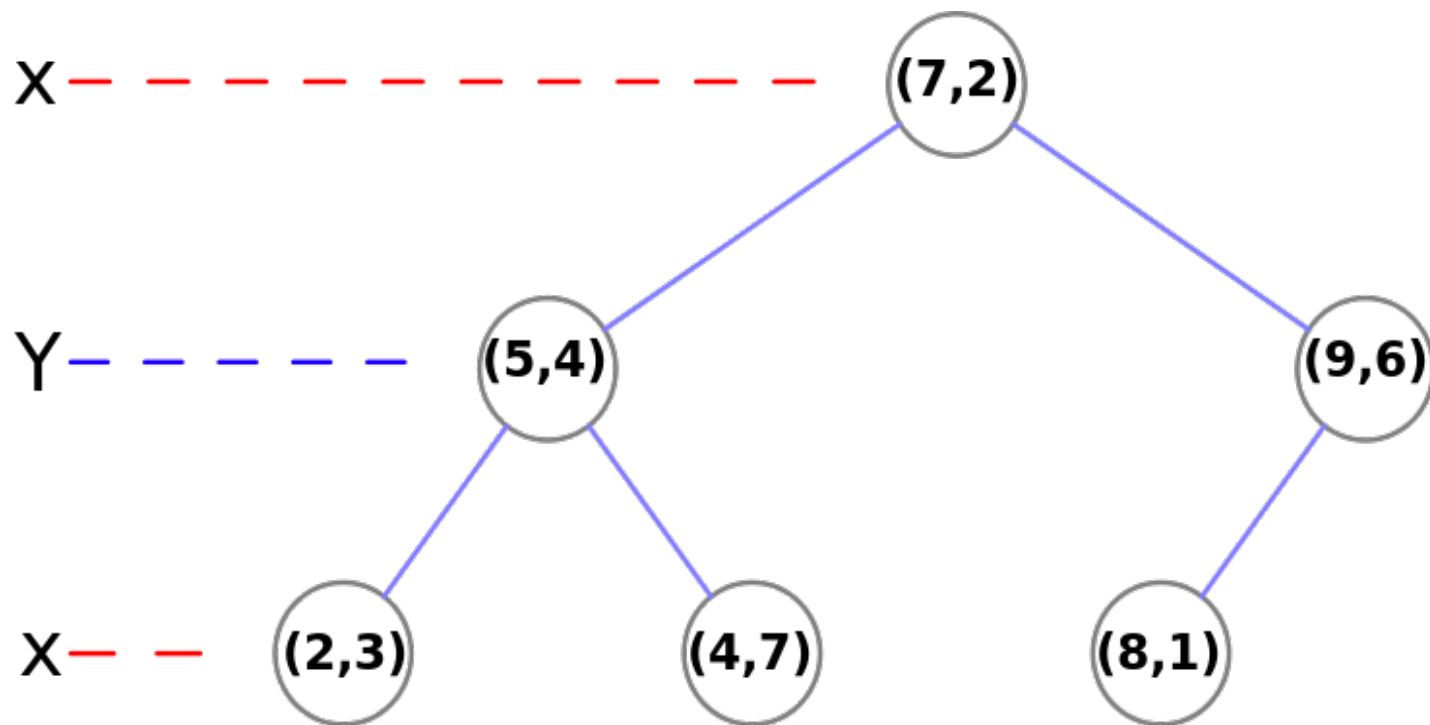


$x = 7$ 将空间划分成左右两个子空间

降低近邻计算

□ KD树构造

✓ 得到KD树:



降低近邻计算

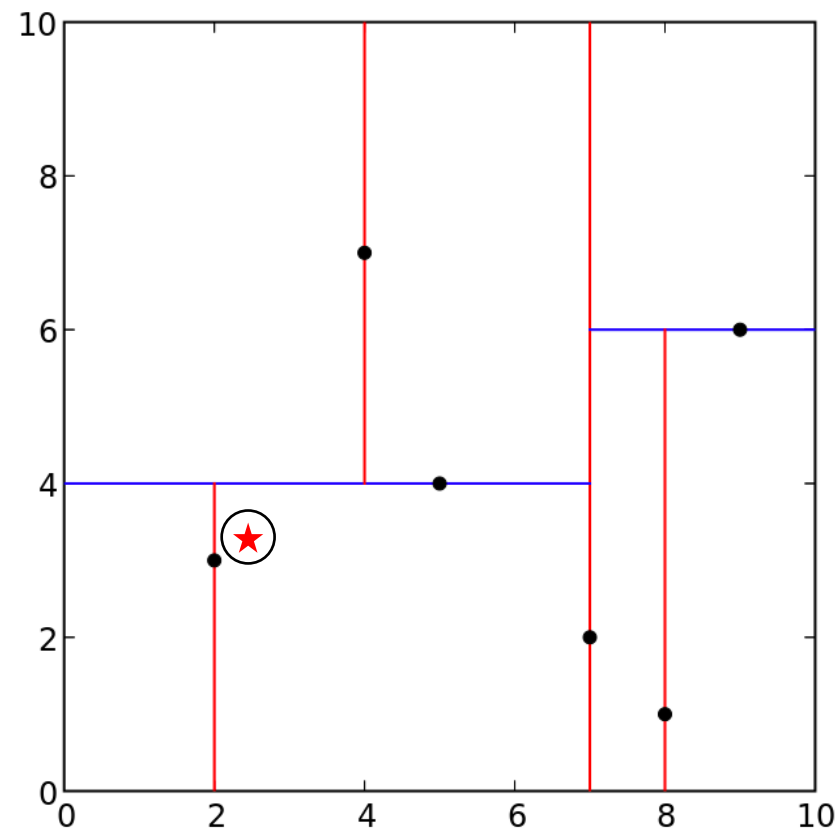
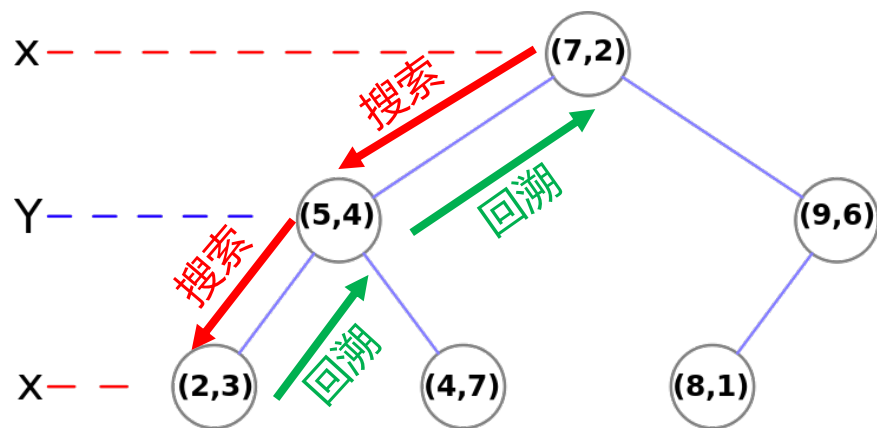
□ KD树搜索

1. 二叉搜索：从根节点开始，以递归的方式从树的顶端向下移动
2. 当到达一个叶子节点，即得到最邻近的近似点，判断其是否为最优，并保存为“当前最优”
3. 回溯：对整棵树进行递归，并对每个节点执行以下操作
 - 如果当前节点比“当前最优”更近，替换为新的“当前最优”
 - 判断分割平面的另一侧是否存在比“当前最优”更优的点。构造一个超球面，球心为查询点，半径为与当前最优的距离
 - ❖ 如果超球面跟超平面相交，则有可能存在更优的点；按照相同的搜索过程，从当前节点向下移动到树的另一个分支以寻找更近的点
 - ❖ 如果超球面跟超平面不相交，则沿着树继续往上走，当前节点的另一个分支则被排除
4. 当算法为根节点完成整个过程时，算法结束

降低近邻计算

□ KD树搜索

1. 查找查询点(2.1,3.1)
2. 二叉搜索，找到叶子节点(2,3)，并且当前最优欧式距离为0.1414
3. 回溯，构造一个超球面（圆）
4. 分别对节点(5,4)和(7,2)进行回溯分析
5. 结束整个搜索，返回最近邻(2,3)



查询点为(2.1,3.1)，图中星号所示

降低近邻计算

□ KD树搜索

✓ 时间复杂度

- $O(n \log^2 n)$, 如果选用复杂度为 $O(n \log n)$ 的排序算法, 如快速排序、堆排序、归并排序来找到中位数
- 为 $O(n \log n)$, 如果选用复杂度为 $O(n)$ median of medians 算法来找到中位数
- . . .
- 寻找最近邻的时间复杂度为: $O(\log n)$

降低近邻计算

□ 降维 (Dimension reduction)

✓ **核心思想**：通过某种数学变换将原始高维属性空间转变为一个低维子空间，来缓解维数灾难问题。

- 多维缩放方法 (Multiple Dimensional Scaling, MDS)
- 主成分分析 (Principal Component Analysis, PCA)
- 局部线性潜在 (Locally Linear Embedding, LLE)
- ISOMAP
-

✓ **推荐阅读**：降维算法相关的书籍和论文

降低近邻计算

□ 降维 (Dimension reduction)

✓ **核心思想**: 搜索可能是近邻的数据项而不再只局限于返回最可能的数据项, 在牺牲可接受范围内的精度的情况下提高检索效率

- 不要求一定是距离最短的k个
- 如第k个最近邻, 其距离是 d_k , 则ANN要求其选取的所有k个样例的距离 $\hat{d} \leq (1 + \varepsilon)d_k$
- 可以将k-NN搜索速度提高几个数量级

✓ **推荐阅读**: FLANN: <https://github.com/mariusmuja/flann>

- ANN相关的软件和论文

降低近邻计算

□ 哈希 (Hashing)

✓ **核心思想**: 利用哈希函数把任意长度的输入映射为固定长度的输出

- Hash函数 f_i : 将 R^d 分为两部分, 分别用 $f_i = 0, 1$ 表示
- 设计 m 个hash函数 $f_1, \dots, f_m$, 每个样本 x 表示为 m 个bit
- $m \ll d$, 计算和存储大幅简化, 需要设计好的hash

✓ **推荐阅读**:

- LSH: https://en.wikipedia.org/wiki/Locality-sensitive_hashing
- Hash function: https://en.wikipedia.org/wiki/Hash_function

大纲

- 回顾
- k -近邻分类器
- 最近邻分类器
- k -近邻回归
- 降低近邻计算
- 扩展阅读

拓展阅读

□ Probabilistic k-NN

- k -NN算法的一个缺点是它不是建立在任何概率框架上
- 无法得到关于类别的后验概率
- 没办法概率化地推断近邻的个数，以及度量的参数
- 可以通过定义似然函数来构造概率化的 k -NN

✓ 推荐阅读：

- <https://www.cc.gatech.edu/~afb/classes/CS7616-Spring2014/slides/CS7616-13a-PKNN.pdf>

拓展阅读

- Boiman O, Shechtman E, Irani M. **In defense of nearest-neighbor based image classification**[C]. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2008: 1-8.
- Li W, Wang L, Xu J, et al. **Revisiting local descriptor based image-to-class measure for few-shot learning**[C]. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 7260-7268.

谢谢!

联系方式: liwenbin@nju.edu.cn

更多信息: <https://cs.nju.edu.cn/liwenbin>

PyTorch入门

助教：邓文骁

2024年03月26日

目录

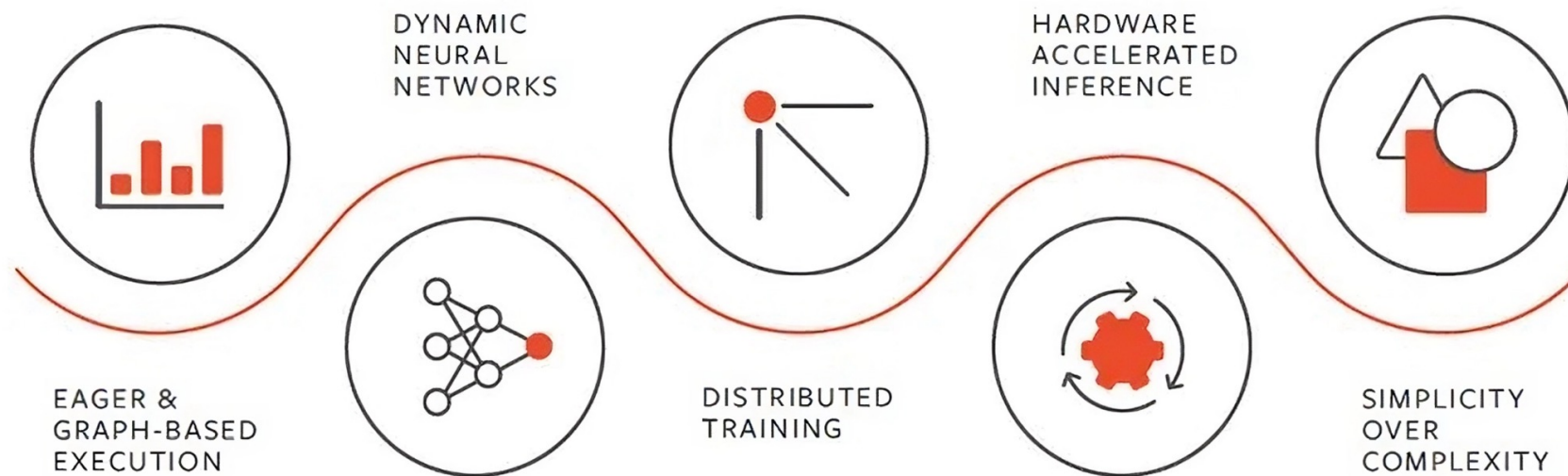
- 介绍与安装
- 基本原理与使用
- 分类示例

目录

- **介绍与安装**
- **基本原理与使用**
- **分类示例**

□ 什么是 PyTorch?

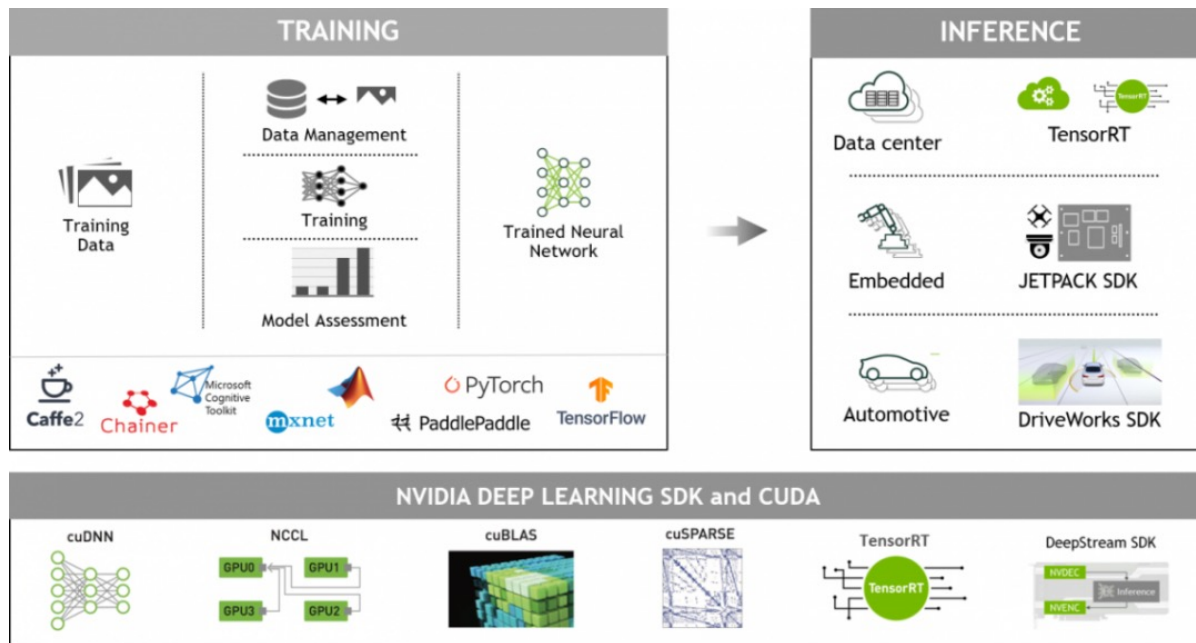
- PyTorch 是 Facebook AI (MetaAI) 开源的深度学习库，它于 2017 年以开源形式发布。
- 作用
 - 该框架将 Torch 中高效而灵活的 GPU 加速后端库与直观的 Python 前端相结合，后者专注于快速原型设计、可读代码，并支持尽可能广泛的深度学习模型。



PyTorch介绍

□ 什么是 PyTorch?

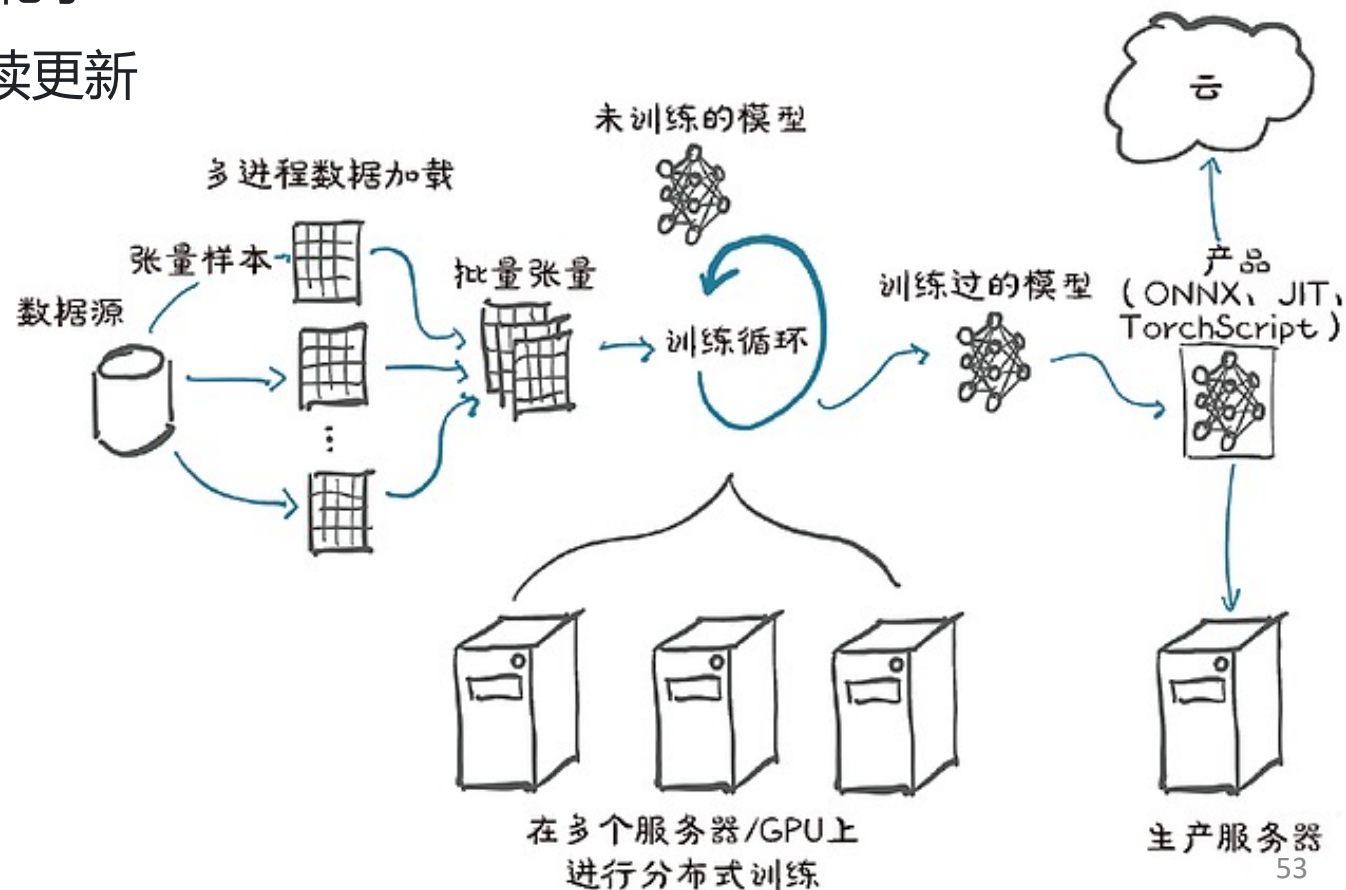
- GPU 加速深度学习框架能够为设计和训练自定义深度神经网络带来灵活性,
- 为 Python 和 C/C++ 等常用编程语言提供编程接口。
- MXNet、PyTorch、TensorFlow 等广泛使用的深度学习框架依赖于 NVIDIA GPU 加速库, 能够提供高性能的多 GPU 加速训练。



介绍

为什么要用pytorch?

- PyTorch算是相当简洁优雅且高效快速的框架
- 设计追求最少的封装，尽量避免重复造轮子
- 大公司的支持足以确保PyTorch获得持续更新
- 不错的文档，作者亲自维护论坛
- 入门简单



深度学习框架

□ 深度学习框架

- [PyTorch](#): tensors and dynamic neural networks in python (Python);
- [TensorFlow](#) : Using data flow graphs (C++, Python);
- [MatConvNet](#): A matlab toolbox implementing CNNs (Matlab);
- [Caffe](#) : C++, Python;
- [Keras](#): A theano based deep learning library;
- [Torch](#): provides a Matlab-like environment for ML algorithms in lua (Lua);
- [Theano](#): CPU/GPU symbolic expression compiler in python (Python);
- [MXNet](#): mix symbolic and imperative programming (Python, R, Julia...);
- [CNTK](#) : a unified deep-learning toolkit by Microsoft (C++, Python).
- [PaddlePaddle](#): An Open-Source Deep Learning Platform (百度)
- [MindSpore](#): 面向“端-边-云”全场景设计的AI框架 (华为)
- [MegEngine](#) : 一个快速、可拓展、易于使用且支持自动求导的深度学习框架 (旷视)
- [Jittor](#): 一个基于即时编译和元算子的高性能深度学习框架 (清华大学)

安装

在Linux系统下安装PyTorch?

- 使用pip安装

PyTorch Build	Stable (2.2.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	ROCm 5.7	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118</pre>			

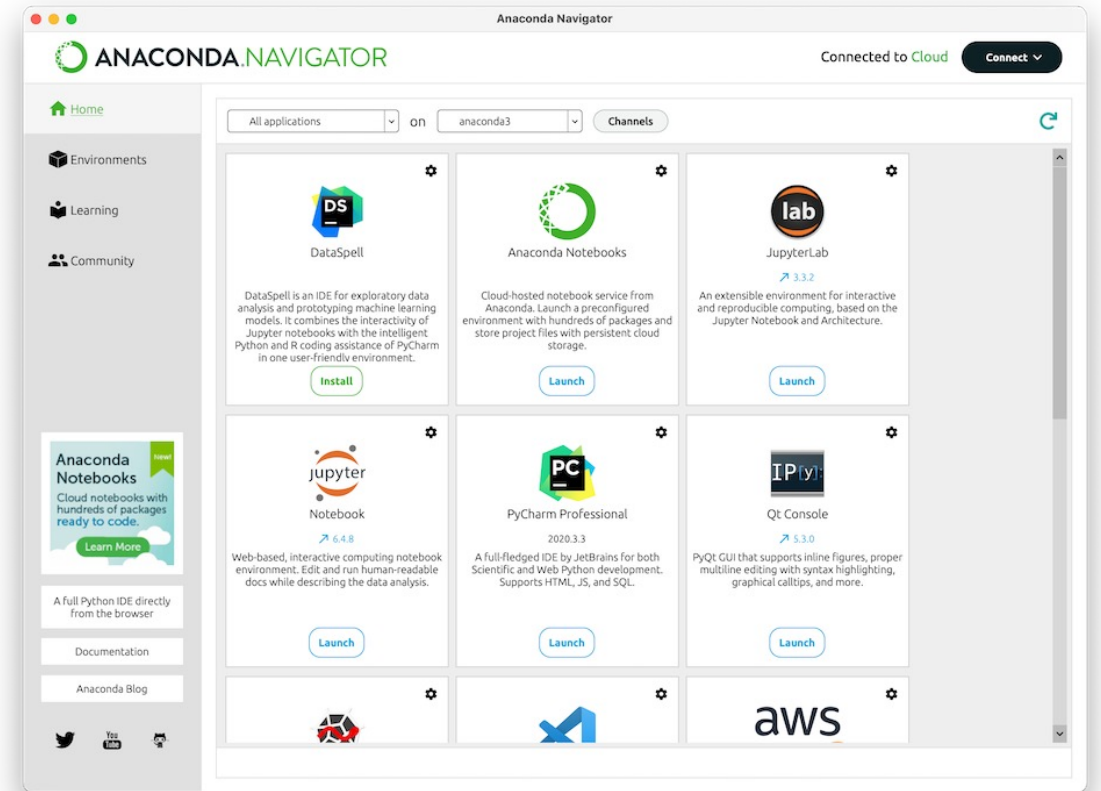
运行该命令行

安装

在Linux系统下安装PyTorch?

- 使用conda安装（环境管理，推荐使用）

PyTorch Build	Stable (2.2.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	ROCm 5.7	CPU
Run this Command:	<pre>conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch -c nvidia</pre>			



安装

注意事项

- 执行完安装命令等待时间长是正常的，因为下载可能比较久
- 尽量保证python版本、cuda驱动版本、pytorch版本之间相互匹配兼容
- 遇到问题上网解答

COMMANDS FOR VERSIONS $\geq$ 1.0.0

v2.2.0

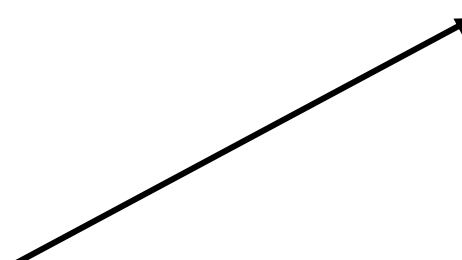
Conda

OSX

```
# conda
conda install pytorch==2.2.0 torchvision==0.17.0 torchaudio==2.2.0 -c pytorch
```

Linux and Windows

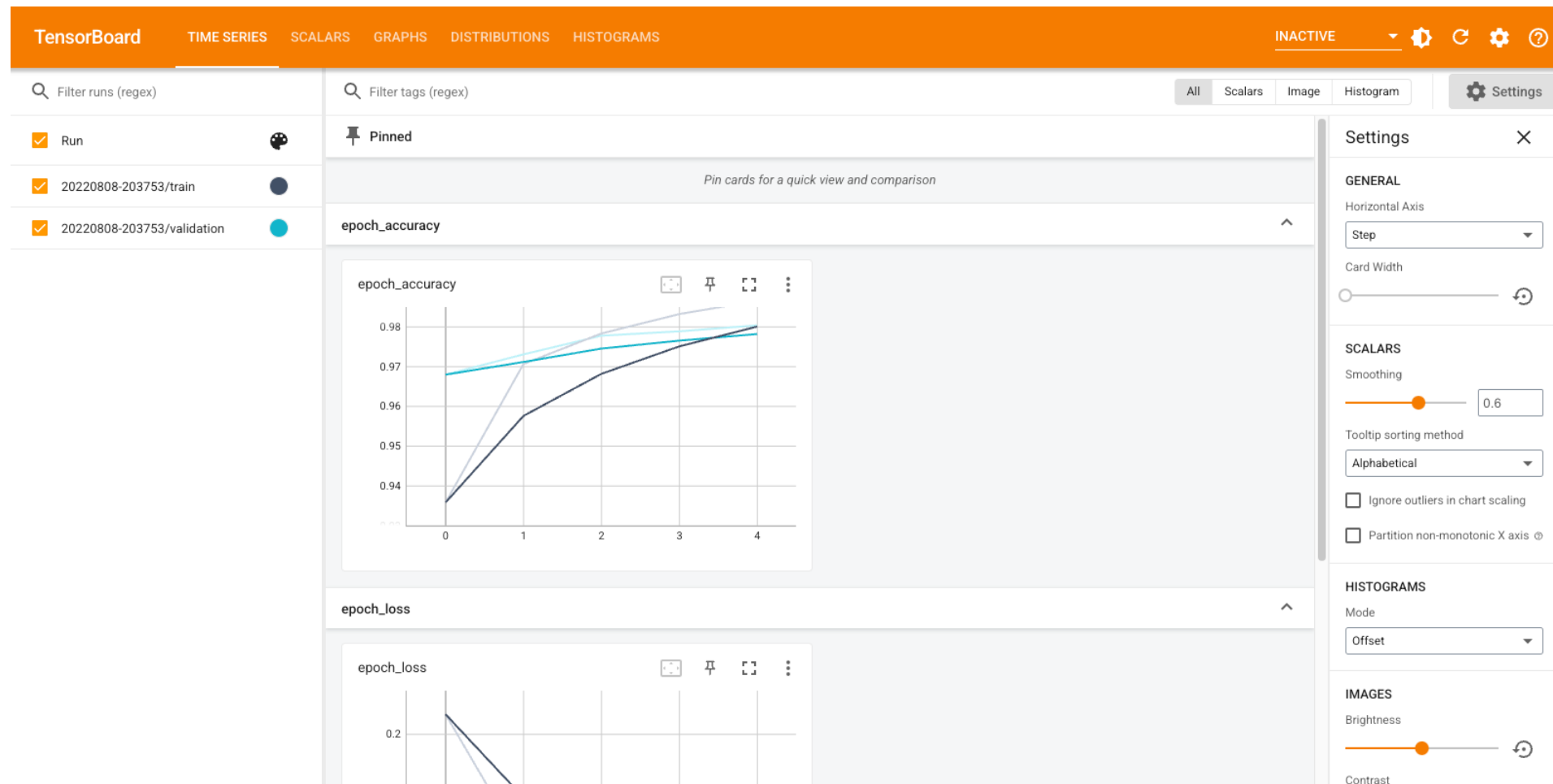
```
# CUDA 11.8
conda install pytorch==2.2.0 torchvision==0.17.0 torchaudio==2.2.0 pytorch-cuda=11.8 -c pytorch -c nvidia
# CUDA 12.1
conda install pytorch==2.2.0 torchvision==0.17.0 torchaudio==2.2.0 pytorch-cuda=12.1 -c pytorch -c nvidia
# CPU Only
conda install pytorch==2.2.0 torchvision==0.17.0 torchaudio==2.2.0 cpuonly -c pytorch
```



历史版本 <https://pytorch.org/get-started/previous-versions/>

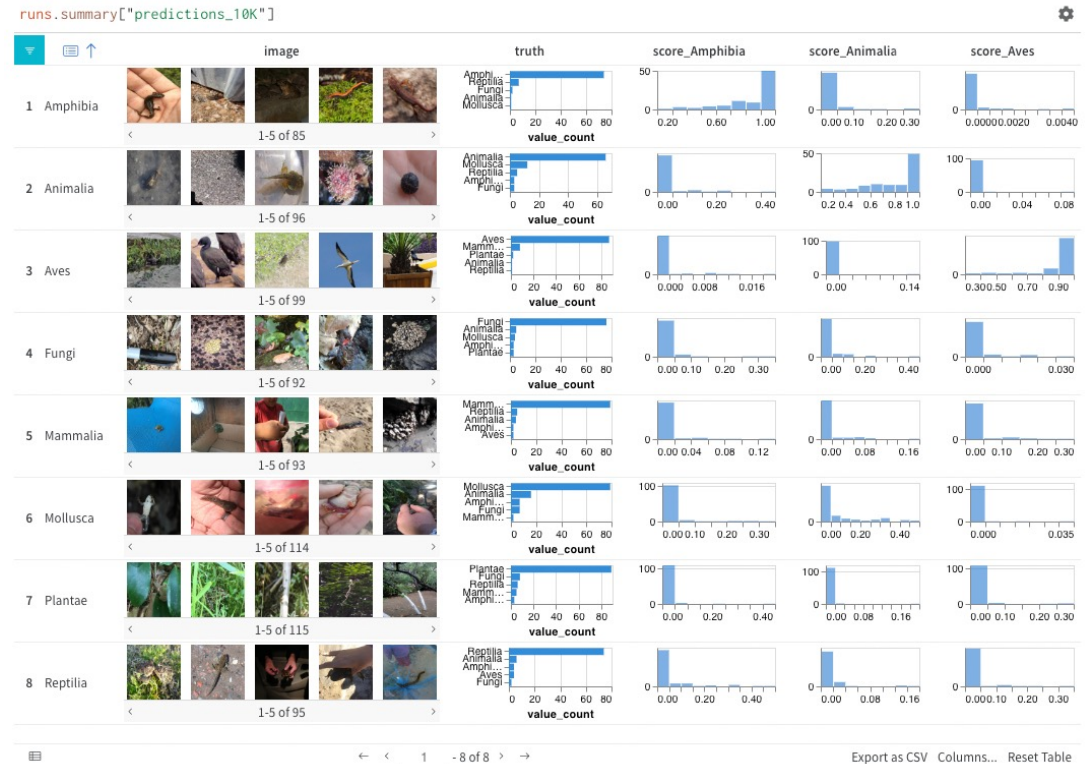
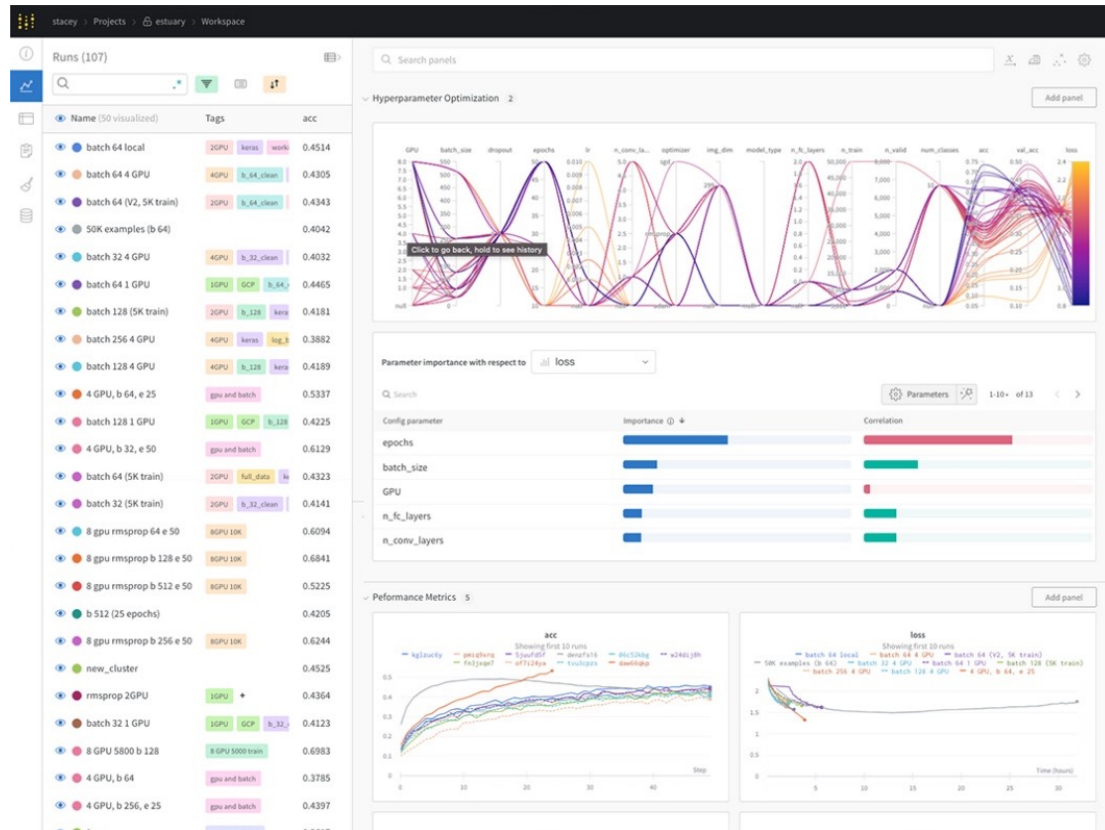
可视化工具TensorBoard

□ Data Flow Graphs



可视化工具Wandb

□ A tool for visualizing and tracking experiments



Wandb网址: <https://github.com/wandb/wandb>

目录

- 介绍与安装
- 基本原理与使用
- 分类示例

Tensor张量

Tensors与Numpy中的 ndarrays类似，但是在PyTorch中 Tensors 可以使用GPU进行计算。

- 张量初始化

创建一个 5x3 矩阵, 但是未初始化:

In [3]:

```
x = torch.empty(5, 3)
print(x)
```

```
tensor([[0.0000, 0.0000, 0.0000],
        [0.0000, 0.0000, 0.0000],
        [0.0000, 0.0000, 0.0000],
        [0.0000, 0.0000, 0.0000],
        [0.0000, 0.0000, 0.0000]])
```

创建一个0填充的矩阵，数据类型为long:

In [5]:

```
x = torch.zeros(5, 3, dtype=torch.long)
print(x)
```

```
tensor([[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]])
```

创建一个随机初始化的矩阵:

In [4]:

```
x = torch.rand(5, 3)
print(x)
```

```
tensor([[0.6972, 0.0231, 0.3087],
        [0.2083, 0.6141, 0.6896],
        [0.7228, 0.9715, 0.5304],
        [0.7727, 0.1621, 0.9777],
        [0.6526, 0.6170, 0.2605]])
```

创建tensor并使用现有数据初始化:

In [6]:

```
x = torch.tensor([5.5, 3])
print(x)
```

```
tensor([5.5000, 3.0000])
```

Tensor张量

- 运算

加法1:

In [9]:

```
y = torch.rand(5, 3)
print(x + y)
```

```
tensor([[ 0.7808, -1.4388,  0.3151],
        [-0.0076,  1.0716, -0.8465],
        [-0.8175,  0.3625, -0.2005],
        [ 0.2435,  0.8512,  0.7142],
        [ 1.4737, -0.8545,  2.4833]])
```

加法2

In [10]:

```
print(torch.add(x, y))
```

```
tensor([[ 0.7808, -1.4388,  0.3151],
        [-0.0076,  1.0716, -0.8465],
        [-0.8175,  0.3625, -0.2005],
        [ 0.2435,  0.8512,  0.7142],
        [ 1.4737, -0.8545,  2.4833]])
```

你可以使用与NumPy索引方式相同的操作来进行对张量的操作

In [13]:

```
print(x[:, 1])
```

```
tensor([-2.0126,  0.4692, -0.5764,  0.6688, -1.1600])
```

`torch.view`: 可以改变张量的维度和大小

译者注: *torch.view* 与Numpy的*reshape*类似

In [14]:

```
x = torch.randn(4, 4)
y = x.view(16)
z = x.view(-1, 8) # size -1 从其他维度推断
print(x.size(), y.size(), z.size())
```

```
torch.Size([4, 4]) torch.Size([16]) torch.Size([2, 8])
```

自动求导机制

- PyTorch 的核心是 autograd 包
- autograd包为张量上的所有操作提供了自动求导。
- 在运行时定义的框架，意味着反向传播是根据你的代码来确定如何运行，每次迭代可以是不同的。

创建一个张量并设置 requires_grad=True 用来追踪他的计算历史

In [3]:

```
x = torch.ones(2, 2, requires_grad=True)
print(x)
```

```
tensor([[1., 1.],
        [1., 1.]], requires_grad=True)
```

对张量进行操作:

In [4]:

```
y = x + 2
print(y)
```

```
tensor([[3., 3.],
        [3., 3.]], grad_fn=<AddBackward0>)
```

结果 `y` 已经被计算出来了，所以，`grad_fn` 已经被自动生成了。

对y进行一个操作

In [6]:

```
z = y * y * 3
out = z.mean()
```

```
print(z, out)
```

```
tensor([[27., 27.],
        [27., 27.]], grad_fn=<MulBackward0>) tensor(27., grad_fn=<MeanBackward0>)
```

自动求导机制

- PyTorch 的核心是 autograd 包
- autograd包为张量上的所有操作提供了自动求导。
- 在运行时定义的框架，意味着反向传播是根据你的代码来确定如何运行，每次迭代可以是不同的。

梯度

反向传播 因为 `out` 是一个纯量 (scalar) , `out.backward()` 等于 `out.backward(torch.tensor(1))` 。

In [8]:

```
out.backward()
```

```
print gradients d(out)/dx
```

In [9]:

```
print(x.grad)
```

```
tensor([[4.5000, 4.5000],  
        [4.5000, 4.5000]])
```

得到矩阵 4.5 .将 `out` 叫做 *Tensor "o"* .

得到 $o = \frac{1}{4} \sum_i z_i$, $z_i = 3(x_i + 2)^2$ 和 $z_i|_{x_i=1} = 27$.

因此, $\frac{\partial o}{\partial x_i} = \frac{3}{2} (x_i + 2)$, 则 $\frac{\partial o}{\partial x_i} \Big|_{x_i=1} = \frac{9}{2} = 4.5$.

神经网络

- 定义一个简单神经网络

开始定义一个网络：

In [1]:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):

    def __init__(self):
        super(Net, self).__init__()
        # 1 input image channel, 6 output channels, 5x5 square convolut
        # kernel
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.conv2 = nn.Conv2d(6, 16, 5)
        # an affine operation:  $y = Wx + b$ 
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        # Max pooling over a (2, 2) window
        x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2))
        # If the size is a square you can only specify a single number
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = x.view(-1, self.num_flat_features(x))
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

    def num_flat_features(self, x):
        size = x.size()[1:] # all dimensions except the batch dimensio
        num_features = 1
        for s in size:
            num_features *= s
        return num_features

net = Net()
print(net)
```

对应网络架构



```
Net(
  (conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (fc1): Linear(in_features=400, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

目录

- 介绍与安装
- 基本原理与使用
- 分类示例

图像分类

- 整体步骤

训练一个图像分类器

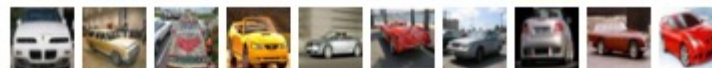
依次按照下列顺序进行：

1. 使用 `torchvision` 加载和归一化CIFAR10训练集和测试集
2. 定义一个卷积神经网络
3. 定义损失函数
4. 在训练集上训练网络
5. 在测试集上测试网络
6. 读取和归一化 CIFAR10

airplane



automobile



bird



cat



deer



dog



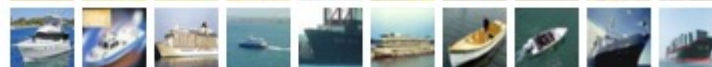
frog



horse



ship



truck



目录

jupyter代码实例讲解